Prueba técnica Data Engineer

Entrega para Pragma

Por: Brayan Estrada

Contenido de la entrega

1	**El repositorio**	Qué hace el pipeline, arquitectura, flujo y decisiones de diseño.
2	**Exploración de los datos**	Estructura, perfil por archivo y hallazgos de calidad en los seis CSV.
3	**Decisiones de limpieza**	Qué pasa con cada caso inválido y dónde se resuelve en el código.
4	**Verificación de resultados**	Se revisan en el notebook docs/notebook/demo_pipeline.ipynb.

Qué hace el pipeline

1

Carga

Lee los cinco CSV (2012-1 … 2012-5) y los inserta en PostgreSQL por micro-batches.

2

Acumula

Mantiene count, avg, min y max de price, actualizados fila a fila y nunca recalculados.

3

Valida

Pasa validation.csv por el mismo flujo, sin código especial.

4

Verifica

Compara las estadísticas en memoria con una consulta directa a la BD, antes y después.

La carpeta docs/ incluye el README, el DDL (schema.md), la exploración de datos, la evidencia de consola y BD, y un notebook de demostración.

Arquitectura del proyecto

```
prueba_tecnica/
├── data/            CSV del reto
├── docs/            docs y evidencia
├── src/
│   ├── main.py      arma el servicio
│   ├── config/      constants, env, logger
│   ├── queries/     todo el SQL
│   ├── resources/   loader, partitioner,
│   │                analyzer, schema, validator
│   ├── services/    PipelineService
│   └── utils/       Decimal, formato
└── tests/           26 tests unitarios
```

Inyección de dependencias

PipelineService recibe todos sus colaboradores por constructor. src/main.py es el único lugar donde se instancian.

SQL centralizado

PipelineQueries concentra todo el SQL. Los recursos no escriben consultas.

El servicio decide qué se registra

Los recursos devuelven datos; el servicio decide qué contar y qué va al log.

Decisiones de diseño: exactitud

Estadísticas incrementales O(1)

Cuatro operaciones por fila: recuento, suma, mínimo y máximo. El promedio sale de suma / recuento. SELECT AVG/MIN/MAX solo se usa para verificar.

Decimal en memoria, NUMERIC en la BD

price se lee como texto y pasa directo a Decimal. El acumulado no arrastra error de float y memoria y BD coinciden exactamente.

Atomicidad por archivo

Los batches de un archivo y el UPSERT de pipeline_stats comparten transacción. Ningún archivo queda a medias; un fallo revierte el archivo entero.

Filas sin precio

Se cargan con price = NULL para conservar el evento, pero update_batch las ignora. La verificación compara count con COUNT(price).

Exploración de los datos

Los seis CSV de data/ antes de diseñar el pipeline

Tres columnas, el mismo formato en los seis archivos

Separador "," · encabezado en la primera línea · codificación UTF-8 · columnas en el mismo orden en todos los archivos

1

`timestamp`

Formato M/D/YYYY sin hora (1/10/2012).

Se parsea con %m/%d/%Y (CSV_TIMESTAMP_FORMAT) para que 5/6/2012 no se lea como 5 de junio.

2

`price`

Enteros positivos escritos como texto.

Algunas celdas vienen vacías. Se lee como texto y se convierte a Decimal, nunca a float.

3

`user_id`

Enteros entre 1 y 10.

Sin nulos en los archivos revisados. Si llegara vacío o no entero, la fila se conserva con NULL.

Los datos están limpios salvo 4 precios vacíos

4

filas sin price

2 en 2012-1.csv y 2 en 2012-4.csv. Sin nulos en timestamp ni user_id.

0

duplicados

Ni filas ni fechas repetidas dentro de un archivo: un evento por día.

0

valores fuera de rango

Sin precios negativos, texto no numérico ni fechas mal formadas.

Orden: los nombres 2012-1 … 2012-5 coinciden con el orden cronológico, así que basta ordenar por nombre; validation.csv es junio y va al final.

Decisiones de limpieza

price vacío o no numérico	Se carga la fila con price = NULL y queda fuera de las estadísticas; el resumen la cuenta como "sin price".	`Validator.validate_df + MetricAnalyzer.update_batch`
timestamp vacío o con otro formato	Se descarta la fila y se registra el motivo en un WARNING.	`Validator.validate_df`
price con decimales	Se conserva exacto como Decimal: se lee como texto, nunca como float.	`CsvLoader.read_batches`

Las filas sin precio se cargan, pero no se imputan

- Se cargan para no perder el registro del evento.
- Cualquier valor inventado alteraría el promedio, el mínimo o el máximo que pide el reto.
- Por eso se ignoran en las estadísticas, y el servicio las cuenta al cerrar cada archivo.

Log al cerrar 2012-1.csv

```
INFO | pipeline_service | Archivo 2012-1.csv cargado: 22 filas (2 sin price, 0 descartadas) |
count=20 avg=59.6500 min=14 max=97
```

La verificación de resultados está en el notebook

- Ejecutar el pipeline paso a paso y en donde se muestra cómo evolucionan las estadísticas tras cada archivo.
- Compara el estado en memoria con COUNT, AVG, MIN y MAX sobre la BD, antes y después de validation.csv.
- La celda con RESET = True reinicia las tablas para correrlo desde cero.

Abrir el notebook

`docs/notebook/demo_pipeline.ipynb`

```
pip install ipykernel    # kernel de Jupyter, no es dependencia del pipeline
```

Gracias

Prueba técnica Data Engineer · Pragma

Brayan Estrada